Imperviousness Classified Change 2015-2018 (raster 20 m)

Author- Ishita Guha Roy

This dataset provides a pan-European level in the spatial resolution of 20 m the degree of imperviousness change in the most relevant categories of sealing change (unchanged no sealing, new cover, loss of cover, unchanged sealed, increased sealing, decreased sealing) between the 2015 and 2018 Imperviousness Density status layers.

This notebook will access the data for Hungary (the country of interest) and see some samples and features of interest.

Understanding Imperviousness

In the Copernicus High Resolution Layer: Imperviousness dataset-

1. **High imperviousness values** → More artificial/sealed surfaces (concrete, asphalt, buildings). These areas don't absorb water well → more runoff, higher flood risk, less groundwater recharge.
2. **Low imperviousness values** → More natural, permeable surfaces (soil, vegetation). These areas can absorb and infiltrate water better → healthier hydrology, less runoff, better for groundwater recharge.

"Low imperviousness" doesn't always mean perfect soil. For example, clay-rich soils might still have poor infiltration despite being "permeable" in land cover terms.

Imperviousness is a **land cover indicator**, not a direct soil quality measure. It mainly reflects human impact on the surface.

Load the Data

In [ ]:
```
import zipfile
#specify path to zip file -

zip_file_path = '/content/IMCC_1518_020m_hu_03035_v010.zip'

# Specify the directory where you want to extract the contents
extract_path = '/content/extracted_files/'
```

```
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)

print(f"Files extracted to: {extract_path}")
```

```
Files extracted to: /content/extracted_files/
```

In [ ]:
```
import os

# List all files in the extracted directory
files = os.listdir(extract_path)
print("Extracted files:", files)
```

```
Extracted files: ['IMCC_1518_020m_hu_03035_v010']
```

In [ ]:
```
# Path to the main extracted folder
main_folder = os.path.join(extract_path, 'IMCC_1518_020m_hu_03035_v010')

# List sub-folders
subfolders = [f for f in os.listdir(main_folder) if os.path.isdir(os.path.joir
print("Sub-folders:", subfolders)

# List files in each sub-folder
for sub in subfolders:
    sub_path = os.path.join(main_folder, sub)
    files = os.listdir(sub_path)
    print(f"\nFiles in {sub}:", files)
```

```
Sub-folders: ['Metadata', 'DATA', 'Symbology']

Files in Metadata: ['IMCC_1518_020m_eu_03035_v010.xml']

Files in DATA: ['IMCC_1518_020m_E49N26_03035_v010.tif', 'IMCC_1518_020m_E51N2
7_03035_v010.tif.vat.dbf', 'IMCC_1518_020m_E47N26_03035_v010.tfw', 'IMCC_1518_0
20m_E51N28_03035_v010.tif.vat.dbf', 'IMCC_1518_020m_E50N27_03035_v010.tif.vat.d
bf', 'IMCC_1518_020m_E49N28_03035_v010.tfw', 'IMCC_1518_020m_E48N27_03035_v01
0.tif', 'IMCC_1518_020m_E52N27_03035_v010.tfw', 'IMCC_1518_020m_E50N27_03035_v0
10.tif', 'IMCC_1518_020m_E49N25_03035_v010.tif', 'IMCC_1518_020m_E50N26_03035_v
010.tif', 'IMCC_1518_020m_E49N25_03035_v010.tfw', 'IMCC_1518_020m_E49N27_0303
5_v010.tif.vat.dbf', 'IMCC_1518_020m_E48N26_03035_v010.tif.vat.dbf', 'IMCC_151
8_020m_E51N26_03035_v010.tif', 'IMCC_1518_020m_E51N26_03035_v010.tfw', 'IMCC_15
18_020m_E51N28_03035_v010.tfw', 'IMCC_1518_020m_E52N27_03035_v010.tif.vat.dbf',
'IMCC_1518_020m_E50N25_03035_v010.tfw', 'IMCC_1518_020m_E50N26_03035_v010.tif.v
at.dbf', 'IMCC_1518_020m_E52N28_03035_v010.tif', 'IMCC_1518_020m_E50N28_03035_v
010.tif.vat.dbf', 'IMCC_1518_020m_E48N26_03035_v010.tfw', 'IMCC_1518_020m_E48N2
5_03035_v010.tif', 'IMCC_1518_020m_E49N27_03035_v010.tfw', 'IMCC_1518_020m_E50N
26_03035_v010.tfw', 'IMCC_1518_020m_E50N25_03035_v010.tif.vat.dbf', 'IMCC_151
8_020m_E48N26_03035_v010.tif', 'IMCC_1518_020m_E48N27_03035_v010.tif.vat.dbf',
'IMCC_1518_020m_E50N27_03035_v010.tfw', 'IMCC_1518_020m_E50N28_03035_v010.tif',
'IMCC_1518_020m_E49N25_03035_v010.tif.vat.dbf', 'IMCC_1518_020m_E49N26_03035_v0
10.tfw', 'IMCC_1518_020m_E52N27_03035_v010.tif', 'IMCC_1518_020m_E50N28_03035_v
010.tfw', 'IMCC_1518_020m_E50N25_03035_v010.tif', 'IMCC_1518_020m_E48N25_0303
5_v010.tif.vat.dbf', 'IMCC_1518_020m_E49N26_03035_v010.tif.vat.dbf', 'IMCC_151
8_020m_E51N27_03035_v010.tif', 'IMCC_1518_020m_E48N27_03035_v010.tfw', 'IMCC_15
18_020m_E48N25_03035_v010.tfw', 'IMCC_1518_020m_E47N26_03035_v010.tif.vat.dbf',
'IMCC_1518_020m_E51N28_03035_v010.tif', 'IMCC_1518_020m_E52N28_03035_v010.tif.v
at.dbf', 'IMCC_1518_020m_E49N28_03035_v010.tif', 'IMCC_1518_020m_E47N26_03035_v
010.tif', 'IMCC_1518_020m_E49N27_03035_v010.tif', 'IMCC_1518_020m_E52N28_0303
5_v010.tfw', 'IMCC_1518_020m_E51N27_03035_v010.tfw', 'IMCC_1518_020m_E49N28_030
35_v010.tif.vat.dbf', 'IMCC_1518_020m_E51N26_03035_v010.tif.vat.dbf']

Files in Symbology: ['IMCC_1518_020m_QGISColour.txt']
```

Access the `GeoTiff` files & see a sample

In [ ]:
```
!pip install rasterio
```

```
Collecting rasterio
  Downloading rasterio-1.4.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x8
6_64.whl.metadata (9.1 kB)
Collecting affine (from rasterio)
  Downloading affine-2.4.0-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-packages
(from rasterio) (25.3.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packag
es (from rasterio) (2025.8.3)
Requirement already satisfied: click>=4.0 in /usr/local/lib/python3.12/dist-pac
kages (from rasterio) (8.2.1)
Collecting cligj>=0.5 (from rasterio)
  Downloading cligj-0.7.2-py3-none-any.whl.metadata (5.0 kB)
Requirement already satisfied: numpy>=1.24 in /usr/local/lib/python3.12/dist-pa
ckages (from rasterio) (2.0.2)
Collecting click-plugins (from rasterio)
  Downloading click_plugins-1.1.1.2-py2.py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-pack
ages (from rasterio) (3.2.3)
Downloading rasterio-1.4.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x8
6_64.whl (22.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22.3/22.3 MB 109.8 MB/s eta 0:00:00
Downloading cligj-0.7.2-py3-none-any.whl (7.1 kB)
Downloading affine-2.4.0-py3-none-any.whl (15 kB)
Downloading click_plugins-1.1.1.2-py2.py3-none-any.whl (11 kB)
Installing collected packages: cligj, click-plugins, affine, rasterio
Successfully installed affine-2.4.0 click-plugins-1.1.1.2 cligj-0.7.2 rasteri
o-1.4.3
```

In [ ]:
```python
# Access the .tif files

import rasterio
data_folder = os.path.join(main_folder, "DATA")

# List all .tif files (ignore .vat.dbf and other auxiliary files)
tif_files = [f for f in os.listdir(data_folder) if f.endswith(".tif")]

print("GeoTIFF files:", tif_files)
```

```
GeoTIFF files: ['IMCC_1518_020m_E49N26_03035_v010.tif', 'IMCC_1518_020m_E48N2
7_03035_v010.tif', 'IMCC_1518_020m_E50N27_03035_v010.tif', 'IMCC_1518_020m_E49N
25_03035_v010.tif', 'IMCC_1518_020m_E50N26_03035_v010.tif', 'IMCC_1518_020m_E51
N26_03035_v010.tif', 'IMCC_1518_020m_E52N28_03035_v010.tif', 'IMCC_1518_020m_E4
8N25_03035_v010.tif', 'IMCC_1518_020m_E48N26_03035_v010.tif', 'IMCC_1518_020m_E
50N28_03035_v010.tif', 'IMCC_1518_020m_E52N27_03035_v010.tif', 'IMCC_1518_020
m_E50N25_03035_v010.tif', 'IMCC_1518_020m_E51N27_03035_v010.tif', 'IMCC_1518_02
0m_E51N28_03035_v010.tif', 'IMCC_1518_020m_E49N28_03035_v010.tif', 'IMCC_1518_0
20m_E47N26_03035_v010.tif', 'IMCC_1518_020m_E49N27_03035_v010.tif']
```

In [ ]:
```python
# Example- read the first GeoTiff File
tif_path = os.path.join(data_folder, tif_files[0])


with rasterio.open(tif_path) as src:
```

```
  print("CRS:", src.crs)
  print("Bounds:", src.bounds)
  print("Number of bands:", src.count)

 # Read first band
  array = src.read(1)
  print("Array shape:", array.shape)
```

```
CRS: IGNF:ETRS89LAEA
Bounds: BoundingBox(left=4900000.0, bottom=2600000.0, right=5000000.0, top=2700
000.0)
Number of bands: 1
Array shape: (5000, 5000)
```

Details of the Features-

Each raster file (GeoTIFF) representing a 5000×5000 grid. Each cell in the array corresponds to a 20 m × 20 m area on the ground, and the values in the array are the **“Imperviousness Classified Change”** codes for 2015–2018.

So in this dataset:

- Array shape (5000, 5000) → the raster has 5000 rows × 5000 columns (cells).
- Single band → each cell has one value, representing the imperviousness class or change class.
- CRS → IGNF:ETRS89LAEA is the projection used.
- Bounds → these are the coordinates in the CRS covering the raster area.

What the array represents:

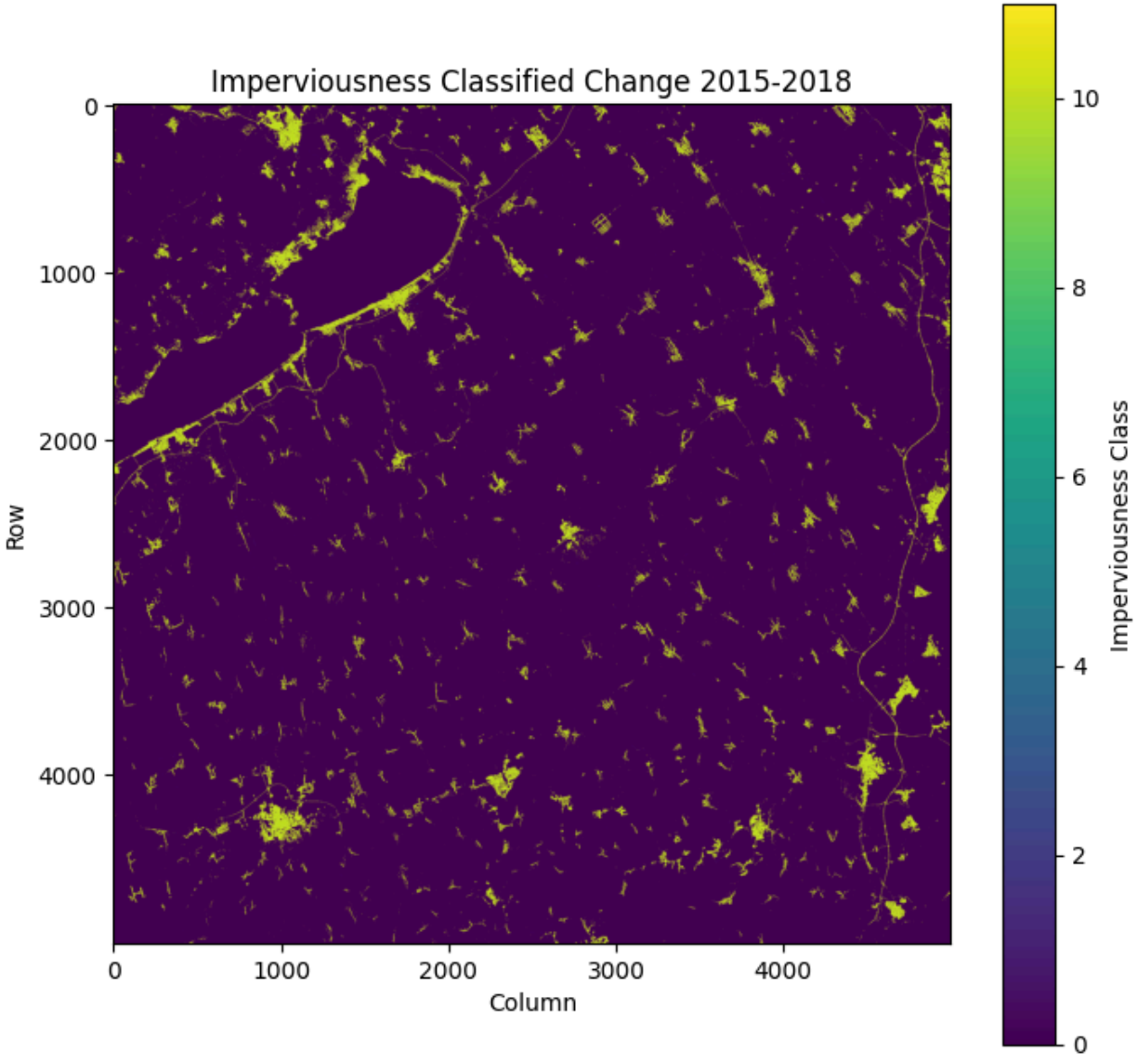
- Each value in the array is a categorical class (e.g., 0 = no imperviousness, 1 = low, …, 100 = fully impervious).
- These are effectively the features, but they are spatially gridded, so instead of rows in a table, we have a 2D spatial grid.

In [ ]:
```
## Look at quick statistics for the first file - tells the range of imperviousness
import numpy as np

print("Min Value:", np.min(array))
print("Max Value:", np.max(array))
print("Mean Value:", np.mean(array))
```

```
Min Value: 0
Max Value: 11
Mean Value: 0.43016692
```

In [ ]:

```python
# Visualize the rastar

import matplotlib.pyplot as plt

plt.figure(figsize=(8,8))
plt.imshow(array, cmap = 'viridis')
plt.colorbar(label = "Imperviousness Class")
plt.title('Imperviousness Classified Change 2015-2018')
plt.xlabel('Column')
plt.ylabel('Row')
plt.show()
```



Visualize all the .tiff files

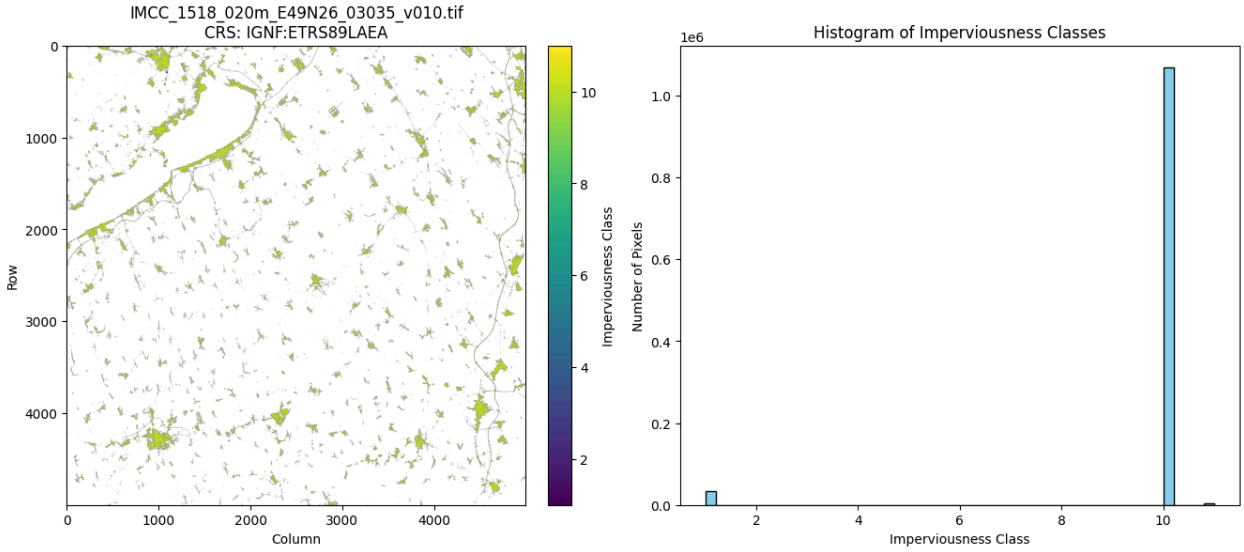
- Load all .tif files from your Hungary DATA folder.
- Plot each raster.
- Plot a histogram of imperviousness classes for each file.

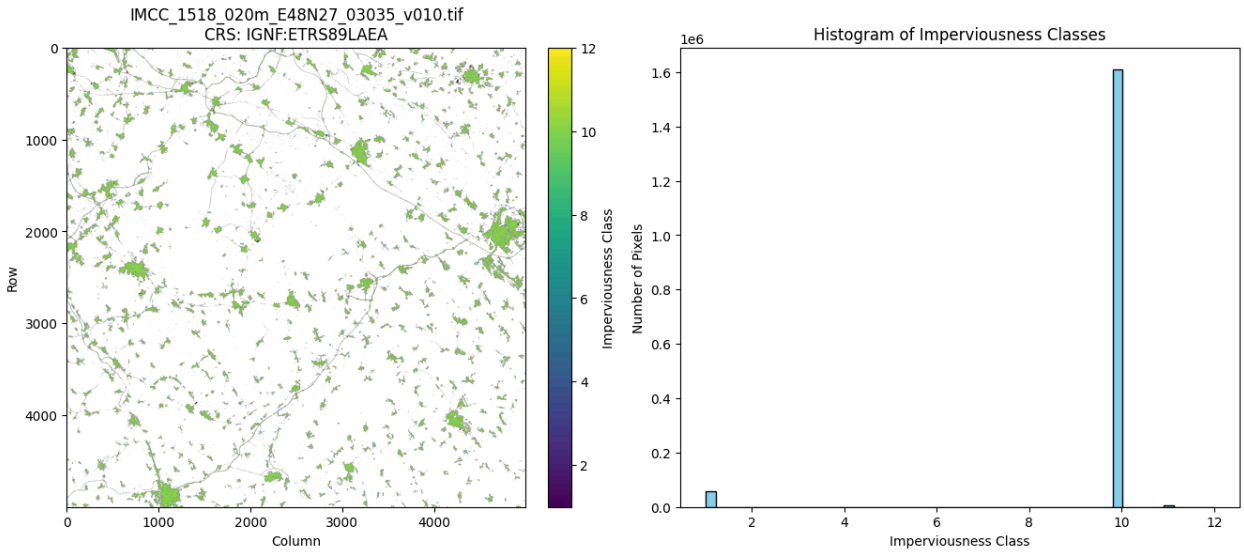
In [ ]:

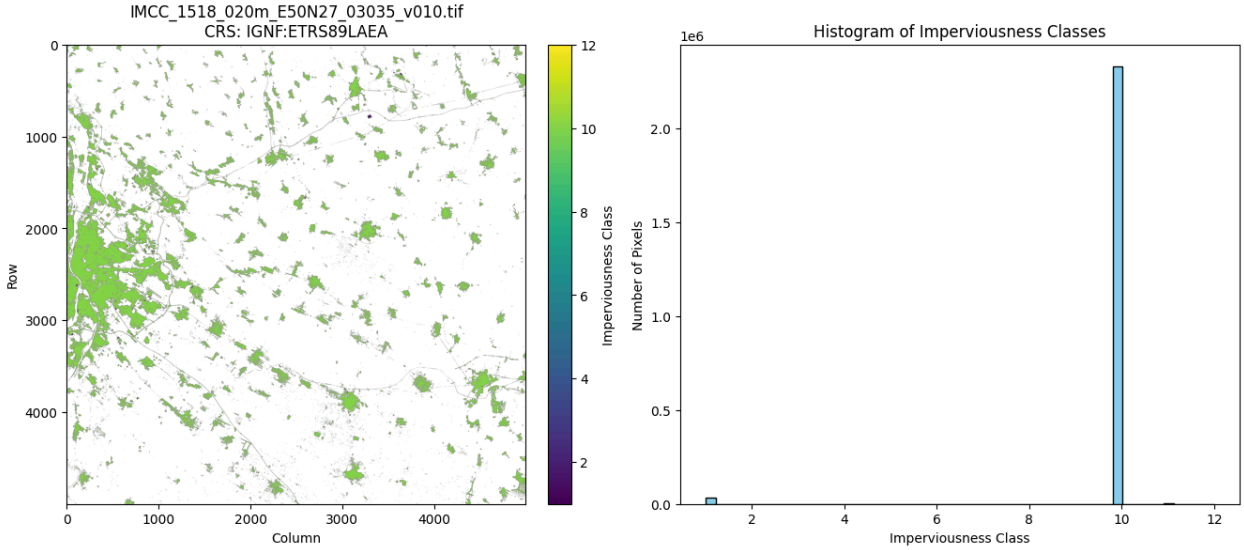
```
import os
import rasterio
import numpy as np
import matplotlib.pyplot as plt

# Path to DATA folder
data_folder = '/content/extracted_files/IMCC_1518_020m_hu_03035_v010/DATA'

# List all .tif files
tif_files = [f for f in os.listdir(data_folder) if f.endswith('.tif')]
print("GeoTIFF files found:", tif_files)

for tif_file in tif_files:
    tif_path = os.path.join(data_folder, tif_file)

    with rasterio.open(tif_path) as src:
        array = src.read(1)
        crs = src.crs
        bounds = src.bounds

        # Mask NoData values (commonly 0)
        array_masked = np.ma.masked_where(array == 0, array)

        #Create subplots- 1 row, 2 columns
        fig, axes = plt.subplots(1, 2, figsize=(14, 6))

        # Left side- Raster plot
        im = axes[0].imshow(array_masked, cmap='viridis')
        fig.colorbar(im, ax=axes[0], fraction=0.046, pad=0.04,
                     label='Imperviousness Class')
        axes[0].set_title(f'{tif_file}\nCRS: {crs}')
        axes[0].set_xlabel('Column')
        axes[0].set_ylabel('Row')


        # Right side- Histogram
        axes[1].hist(array_masked.compressed(), bins = 50,
                     color = 'skyblue', edgecolor = 'black')
        axes[1].set_title('Histogram of Imperviousness Classes')
        axes[1].set_xlabel('Imperviousness Class')
        axes[1].set_ylabel('Number of Pixels')

        # Layout and show
```

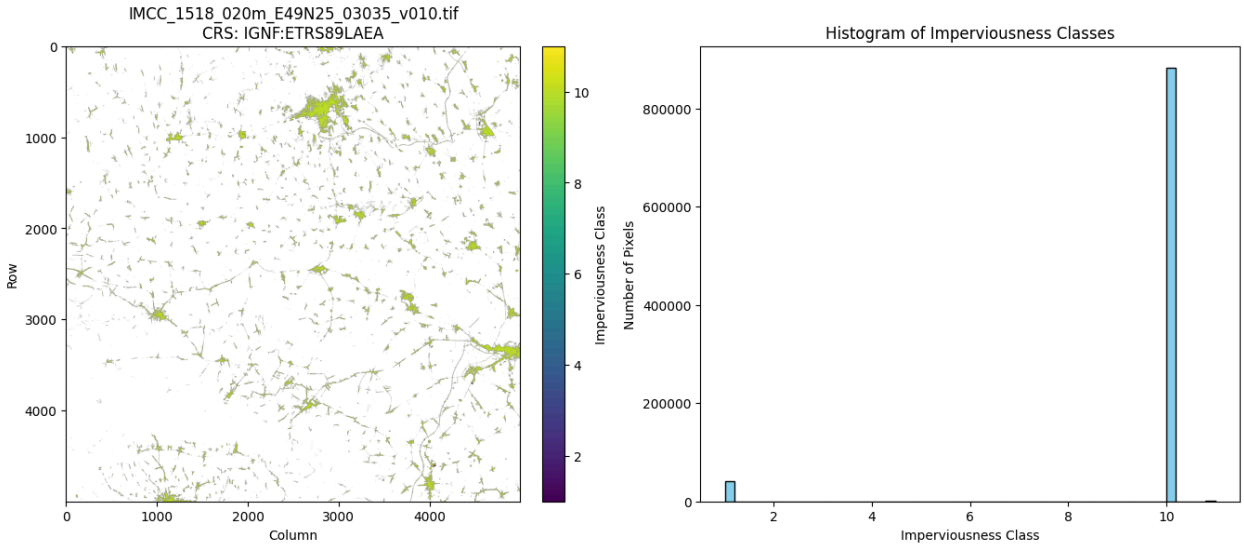
```
        plt.tight_layout()
        plt.show()
```

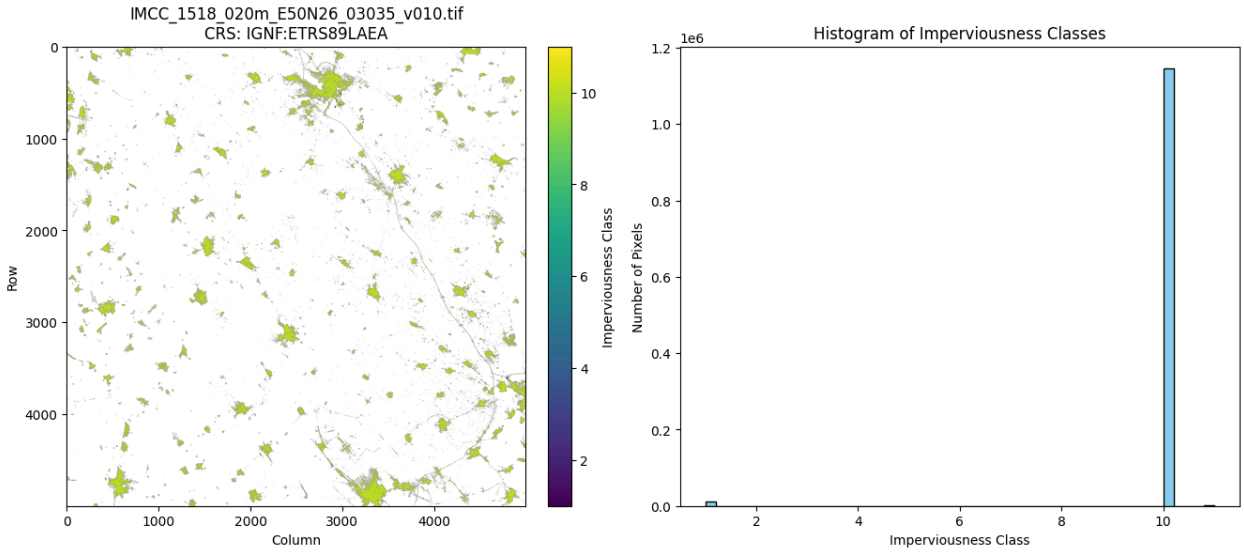
```
GeoTIFF files found: ['IMCC_1518_020m_E49N26_03035_v010.tif', 'IMCC_1518_020m_E
48N27_03035_v010.tif', 'IMCC_1518_020m_E50N27_03035_v010.tif', 'IMCC_1518_020
m_E49N25_03035_v010.tif', 'IMCC_1518_020m_E50N26_03035_v010.tif', 'IMCC_1518_02
0m_E51N26_03035_v010.tif', 'IMCC_1518_020m_E52N28_03035_v010.tif', 'IMCC_1518_0
20m_E48N25_03035_v010.tif', 'IMCC_1518_020m_E48N26_03035_v010.tif', 'IMCC_151
8_020m_E50N28_03035_v010.tif', 'IMCC_1518_020m_E52N27_03035_v010.tif', 'IMCC_15
18_020m_E50N25_03035_v010.tif', 'IMCC_1518_020m_E51N27_03035_v010.tif', 'IMCC_1
518_020m_E51N28_03035_v010.tif', 'IMCC_1518_020m_E49N28_03035_v010.tif', 'IMC
C_1518_020m_E47N26_03035_v010.tif', 'IMCC_1518_020m_E49N27_03035_v010.tif']
```

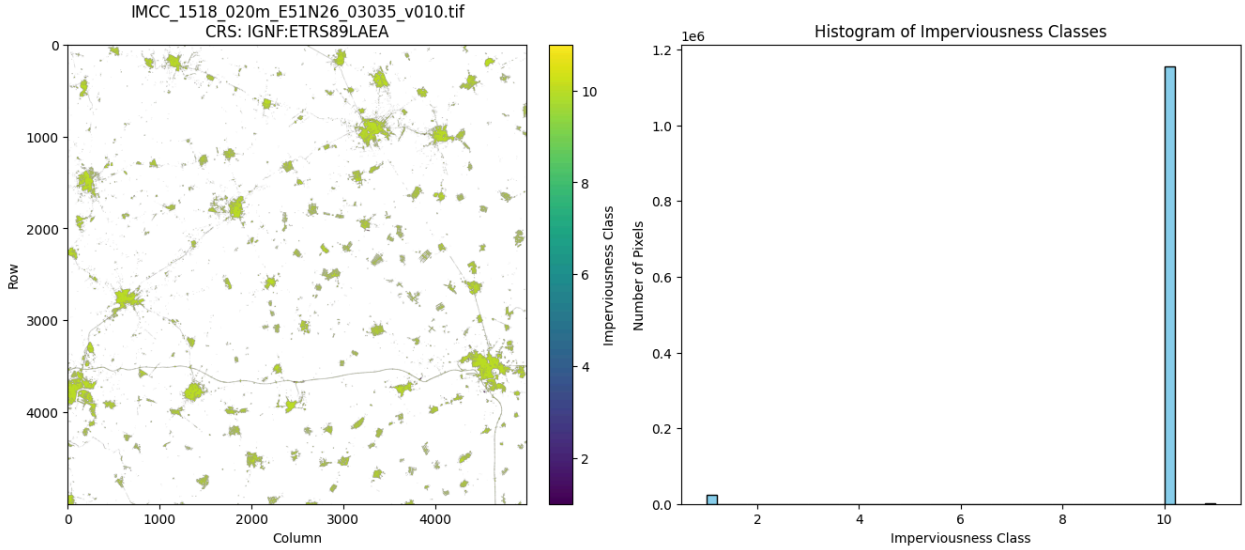


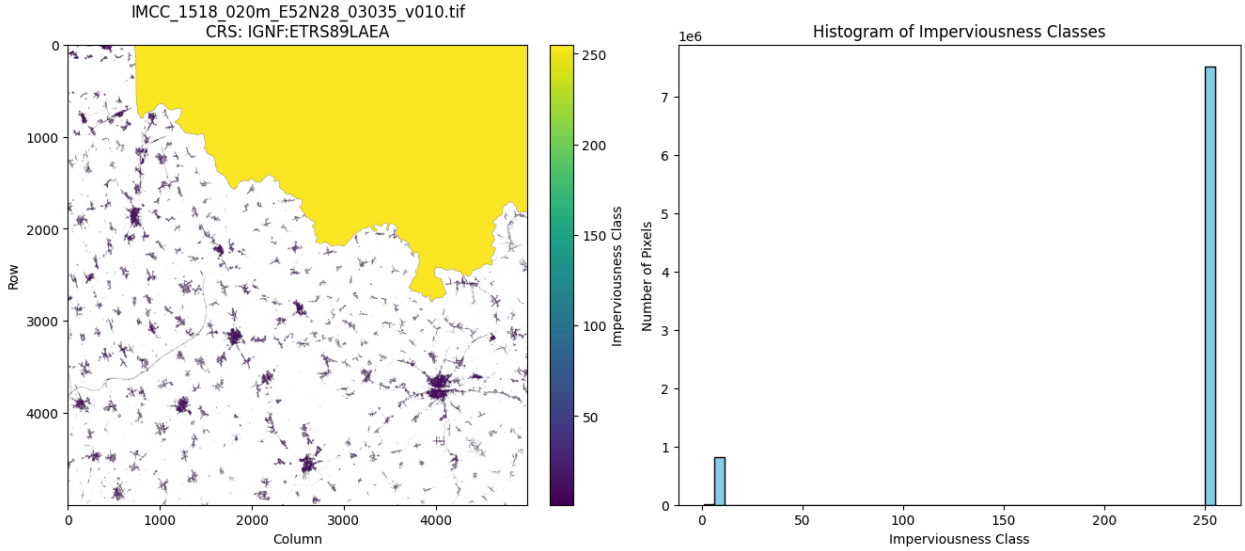


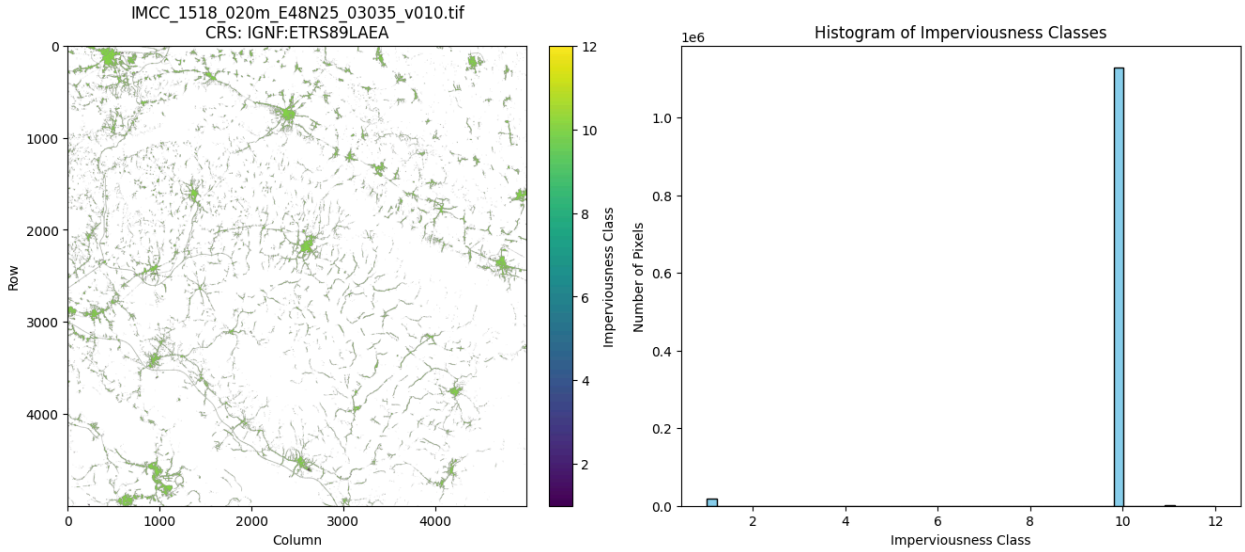

IMCC_1518_020m_E50N27_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class

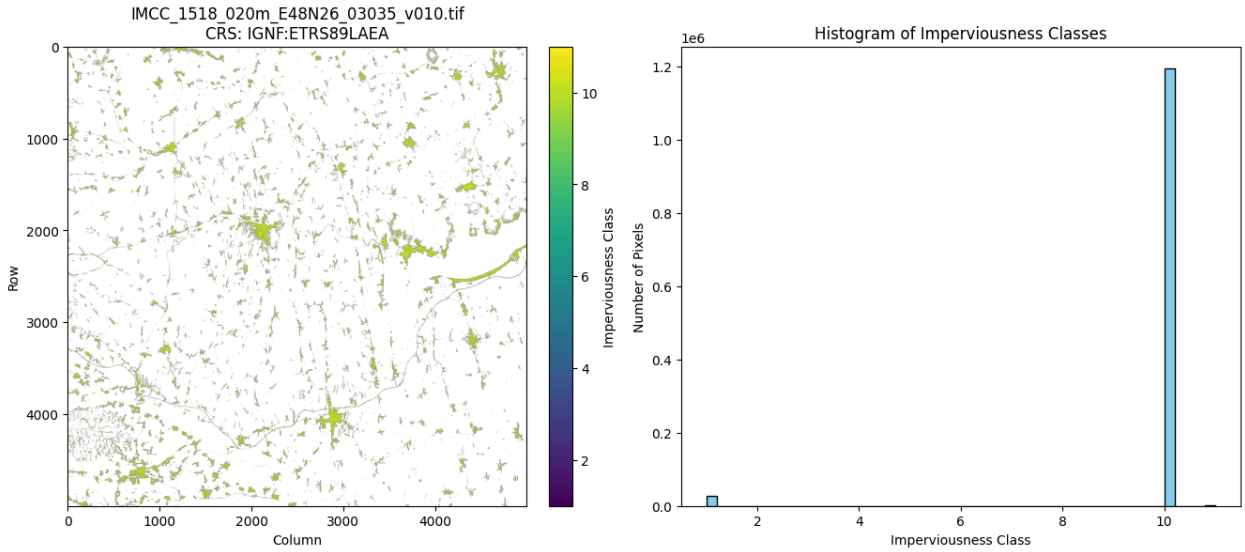

IMCC_1518_020m_E49N25_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
Imperviousness Class

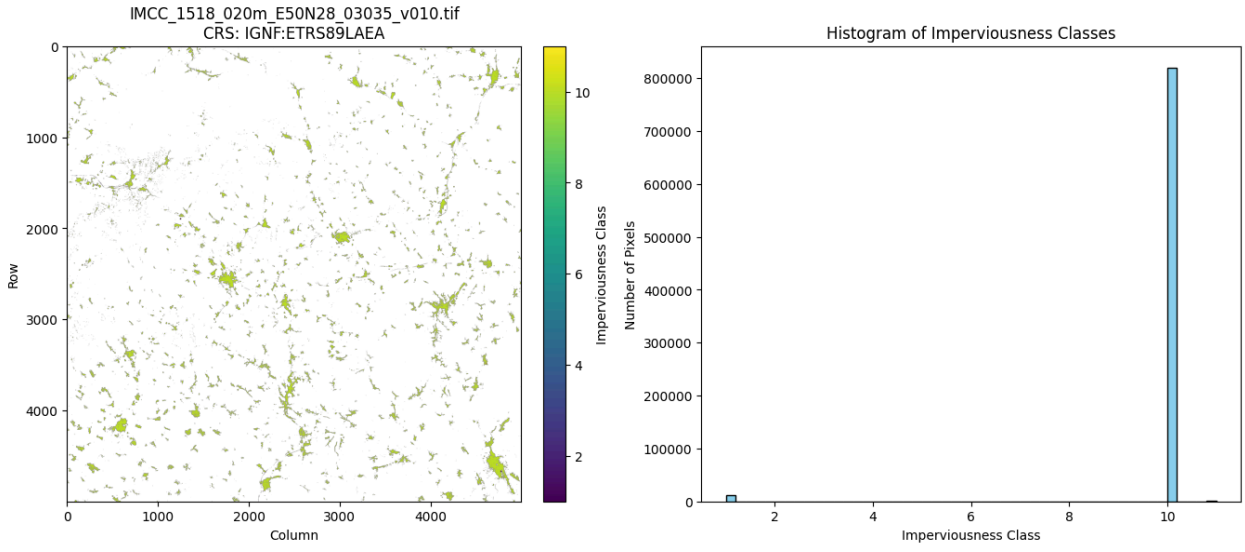

IMCC_1518_020m_E50N26_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class

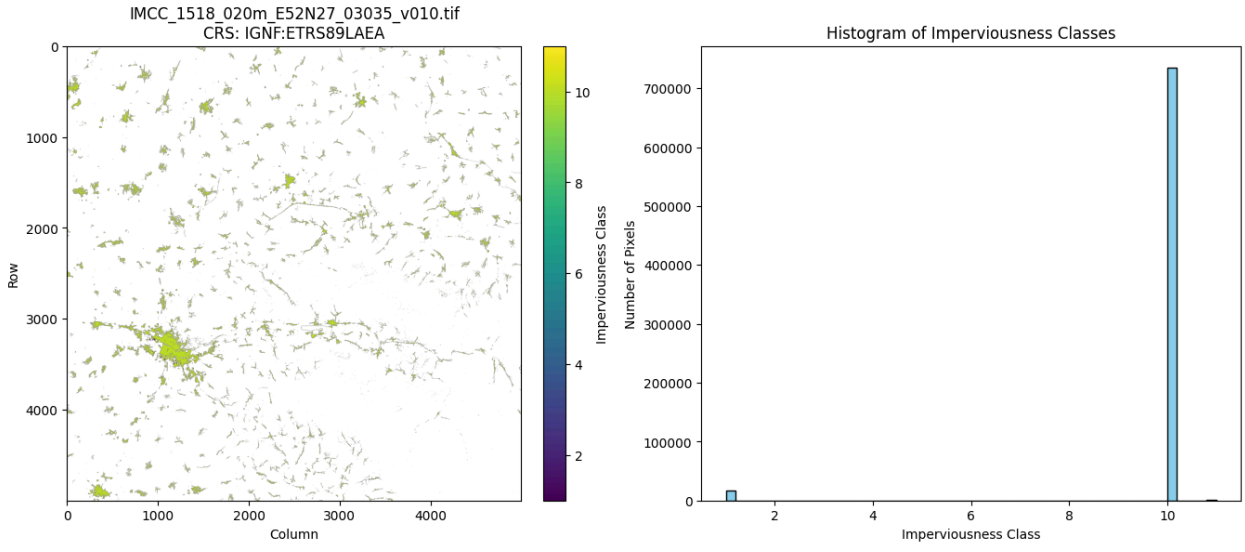

IMCC_1518_020m_E51N26_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class

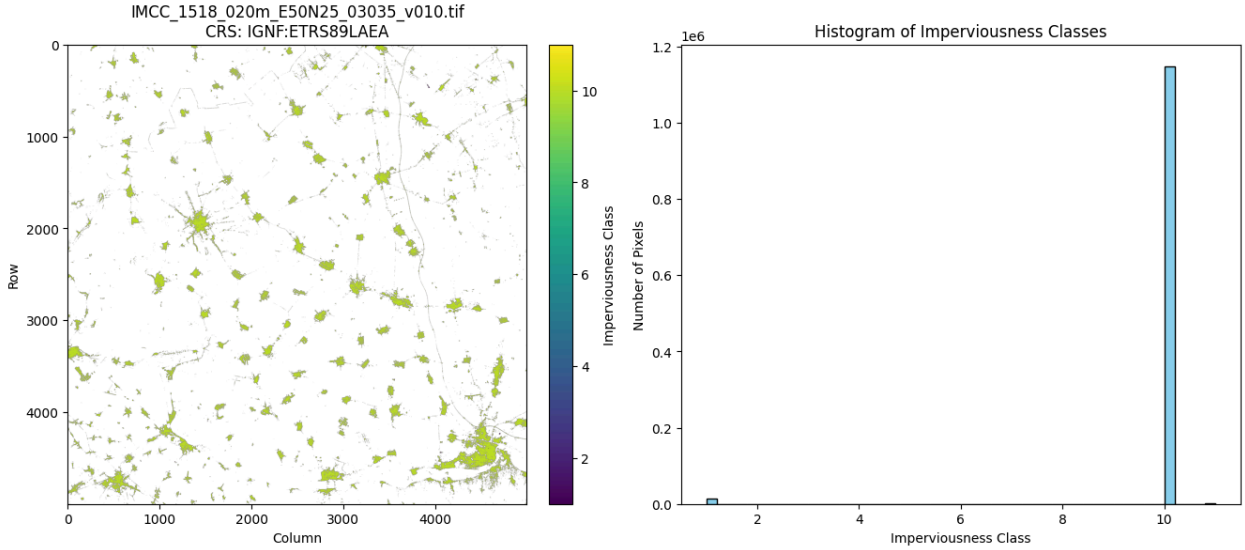

IMCC_1518_020m_E52N28_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class

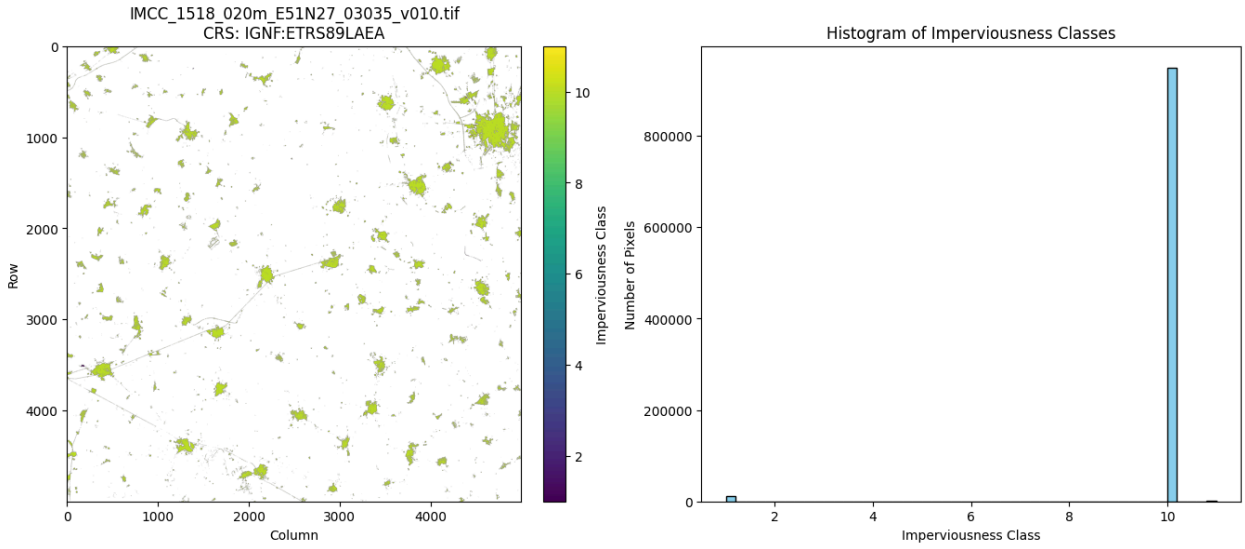

IMCC_1518_020m_E48N25_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class

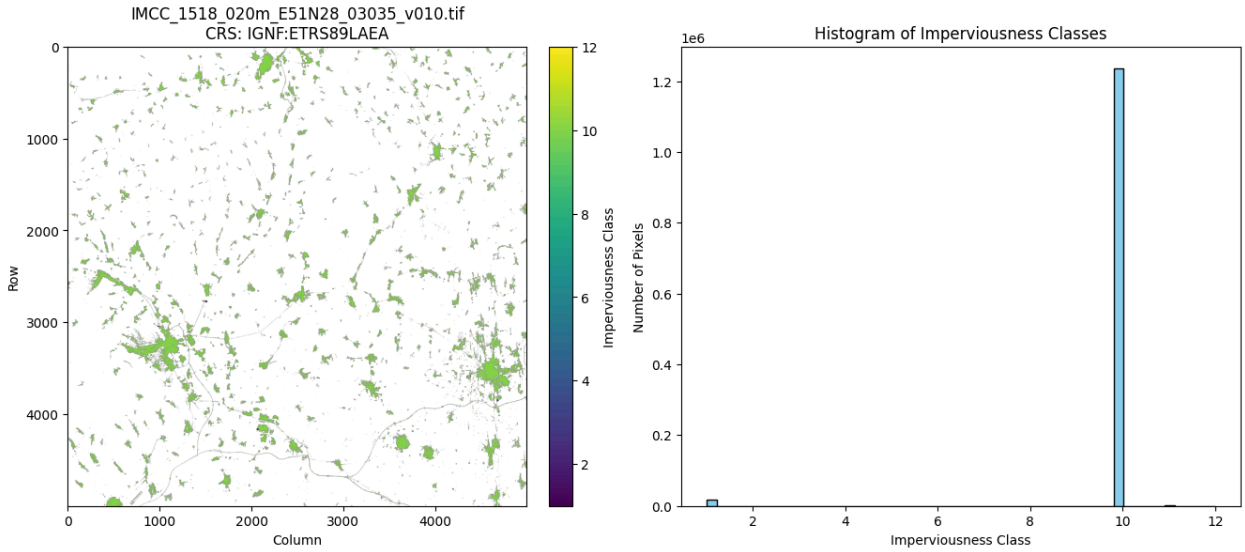

IMCC_1518_020m_E48N26_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class

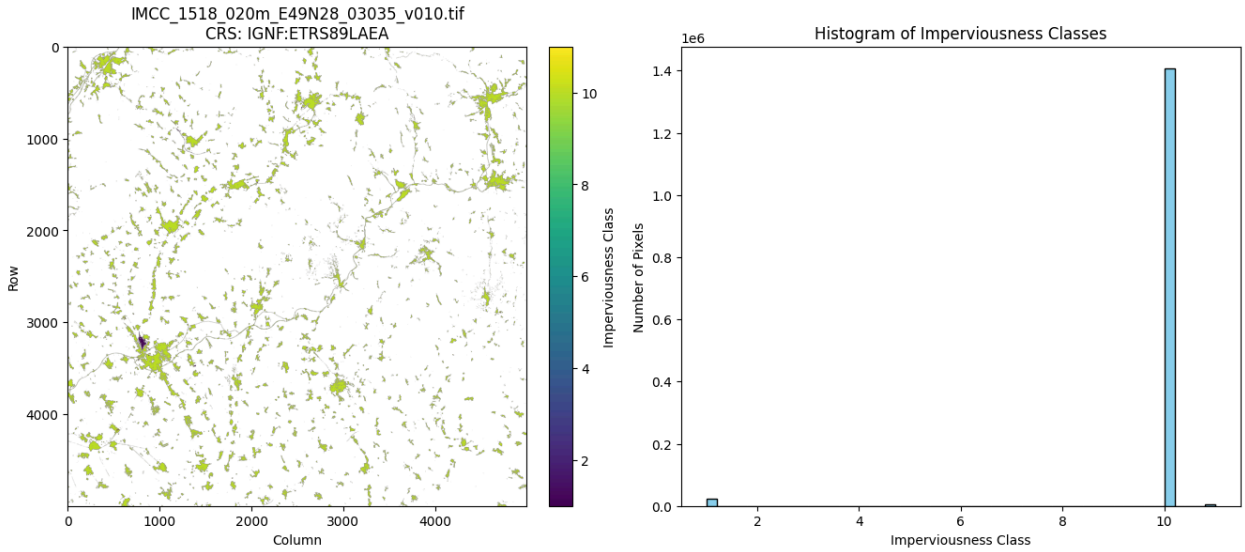

IMCC_1518_020m_E50N28_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
Imperviousness Class

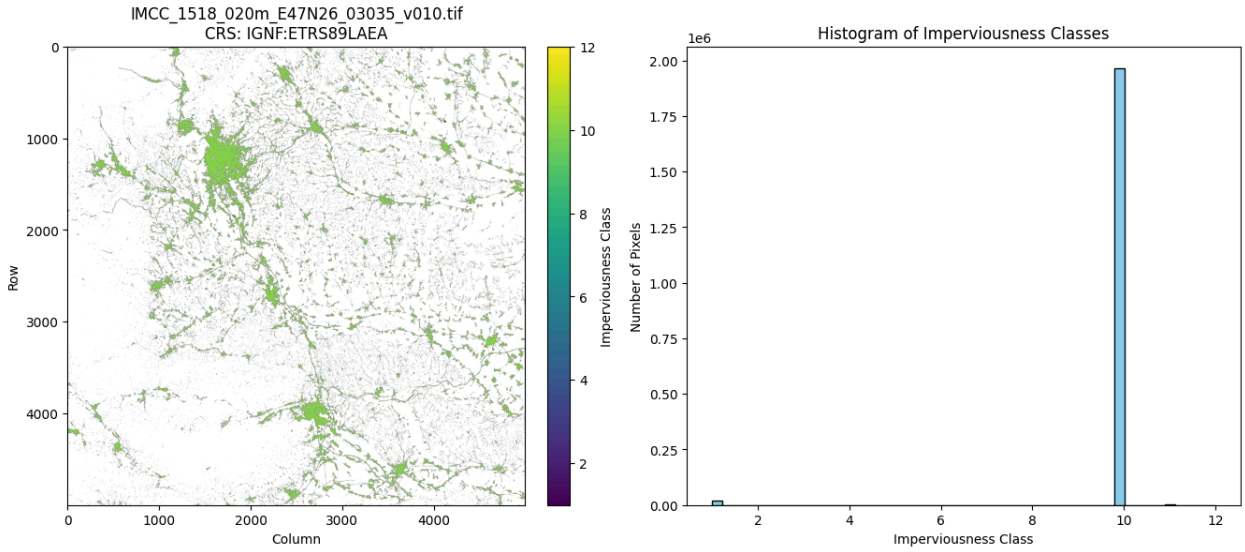

IMCC_1518_020m_E52N27_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
Imperviousness Class

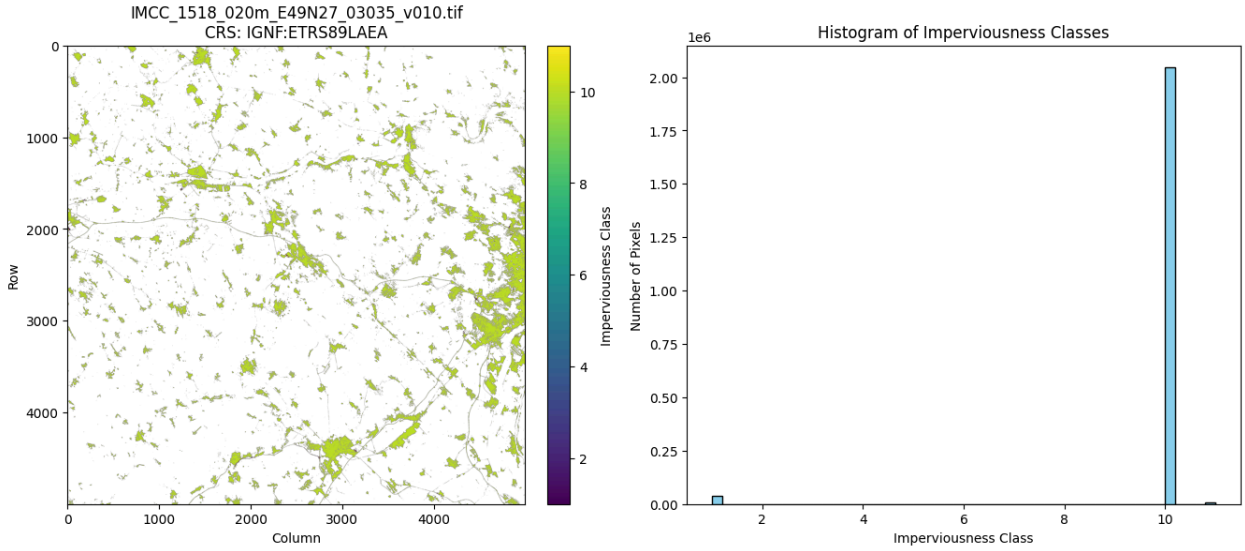

IMCC_1518_020m_E50N25_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class


IMCC_1518_020m_E51N27_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
Imperviousness Class


IMCC_1518_020m_E51N28_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
1e6
Number of Pixels
Imperviousness Class


IMCC_1518_020m_E49N28_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
1e6
Imperviousness Class


IMCC_1518_020m_E47N26_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
1e6
Imperviousness Class


IMCC_1518_020m_E49N27_03035_v010.tif
CRS: IGNF:ETRS89LAEA
Row
Column
Imperviousness Class
Histogram of Imperviousness Classes
Number of Pixels
1e6
Imperviousness Class

Convert Raster to pandas DataFrame

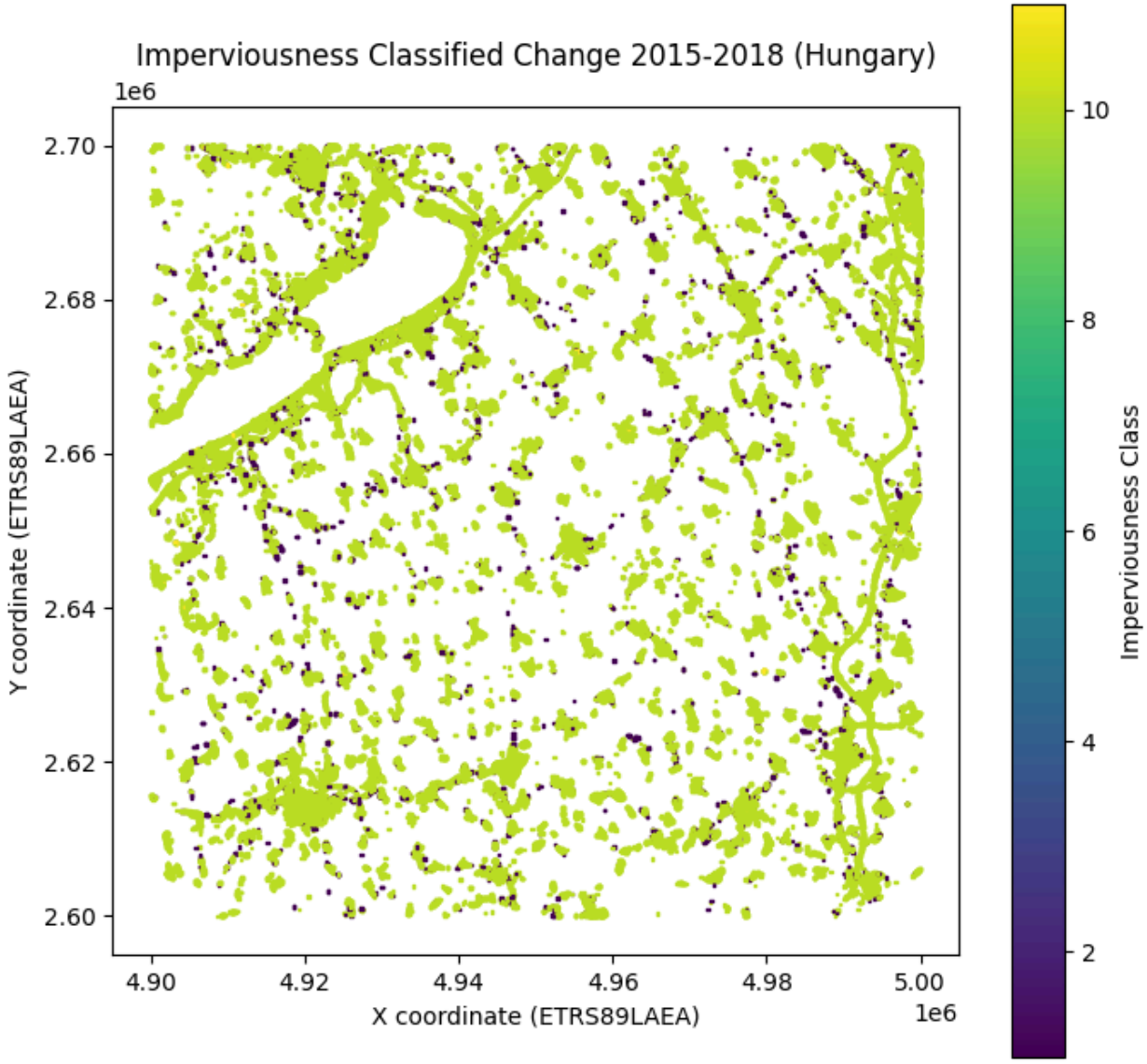
For ease of analysis and ML training, the `.tiff` files can be converted into a `pandas` data frame.

In [ ]:
```
import os
import rasterio
import numpy as np
import pandas as pd

# Path to DATA folder
data_folder = '/content/extracted_files/IMCC_1518_020m_hu_03035_v010/DATA'

# Get the first .tif file
tif_files = [f for f in os.listdir(data_folder) if f.endswith('.tif')]
first_tif = tif_files[0]
tif_path = os.path.join(data_folder, first_tif)
print("Processing file:", first_tif)

# Open the raster
with rasterio.open(tif_path) as src:
    array = src.read(1)           # read first band
    transform = src.transform     # affine transform for coordinates

    # Mask NoData values (commonly 0)
    array_masked = np.ma.masked_where(array == 0, array)

    # Get row and column indices of valid pixels
    rows, cols = np.where(~array_masked.mask)

    # Convert row, col to x, y coordinates
    xs, ys = rasterio.transform.xy(transform, rows, cols)

    # Get imperviousness values
    values = array_masked[rows, cols]

    # Create DataFrame
    df = pd.DataFrame({
        'x': xs,
        'y': ys,
        'imperviousness': values
    })

print("DataFrame created!")
print(df.head())
print("Total pixels:", df.shape[0])
```

```
Processing file: IMCC_1518_020m_E49N26_03035_v010.tif
DataFrame created!
           x          y  imperviousness
0  4900410.0  2699990.0              10
1  4900430.0  2699990.0              10
2  4900450.0  2699990.0              10
3  4900470.0  2699990.0              10
4  4900490.0  2699990.0              10
Total pixels: 1105345
```

In [ ]:

```python
# Optional: downsample for faster plotting if too many pixels
df_plot = df.sample(frac=0.1, random_state=42)  # use 10% of pixels

plt.figure(figsize=(8, 8))
sc = plt.scatter(
    df_plot['x'],
    df_plot['y'],
    c=df_plot['imperviousness'],
    cmap='viridis',
    s=1,      # size of each point
    marker='s'
)
plt.colorbar(sc, label='Imperviousness Class')
plt.title('Imperviousness Classified Change 2015-2018 (Hungary)')
plt.xlabel('X coordinate (ETRS89LAEA)')
plt.ylabel('Y coordinate (ETRS89LAEA)')
plt.gca().set_aspect('equal', adjustable='box')  # equal aspect ratio
plt.show()
```



Convert all GeoTiff features into a DataFrame

In [ ]:
```
#Path to DATA folder

data_folder= '/content/extracted_files/IMCC_1518_020m_hu_03035_v010/DATA'

#List all tiff files
tif_files = [f for f in os.listdir(data_folder) if f.endswith('.tif')]
print("Found tiff Files:", len(tif_files))


#List to hold ALL data
all_dfs = []

for tif_file in tif_files:
```

```
  tif_path = os.path.join(data_folder, tif_file)
  print(f"Processing {tif_file}...")

  with rasterio.open(tif_path) as src:
    array = src.read(1)
    transform = src.transform

    #Mask NoData values commonly 0
    array_masked = np.ma.masked_where(array == 0, array)

    #Get valid pixel indices
    rows, cols = np.where(~array_masked.mask)

    # Convert indices to coordinates
    xs, ys = rasterio.transform.xy(transform, rows, cols)
    values = array_masked[rows, cols]

    # Create dataframe for this raster
    df = pd.DataFrame({
            'x': xs,
            'y': ys,
            'imperviousness': values,
            'source_file': tif_file   # track which tile it came from
        })

    all_dfs.append(df)

# Combine all into one big dataframe
full_df = pd.concat(all_dfs, ignore_index=True)

print("✅ Combined DataFrame created!")
print("Shape:", full_df.shape)
print(full_df.head())
```

```
Found tiff Files: 17
Processing IMCC_1518_020m_E49N26_03035_v010.tif...
Processing IMCC_1518_020m_E48N27_03035_v010.tif...
Processing IMCC_1518_020m_E50N27_03035_v010.tif...
Processing IMCC_1518_020m_E49N25_03035_v010.tif...
Processing IMCC_1518_020m_E50N26_03035_v010.tif...
Processing IMCC_1518_020m_E51N26_03035_v010.tif...
Processing IMCC_1518_020m_E52N28_03035_v010.tif...
Processing IMCC_1518_020m_E48N25_03035_v010.tif...
Processing IMCC_1518_020m_E48N26_03035_v010.tif...
Processing IMCC_1518_020m_E50N28_03035_v010.tif...
Processing IMCC_1518_020m_E52N27_03035_v010.tif...
Processing IMCC_1518_020m_E50N25_03035_v010.tif...
Processing IMCC_1518_020m_E51N27_03035_v010.tif...
Processing IMCC_1518_020m_E51N28_03035_v010.tif...
Processing IMCC_1518_020m_E49N28_03035_v010.tif...
Processing IMCC_1518_020m_E47N26_03035_v010.tif...
Processing IMCC_1518_020m_E49N27_03035_v010.tif...
✅ Combined DataFrame created!
Shape: (29616769, 4)
```

	x	y	imperviousness	source_file
0	4900410.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
1	4900430.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
2	4900450.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
3	4900470.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
4	4900490.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif

In [ ]:
```
full_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29616769 entries, 0 to 29616768
Data columns (total 4 columns):
 #   Column          Dtype  
---  ------          -----  
 0   x               float64
 1   y               float64
 2   imperviousness  uint8  
 3   source_file     object 
dtypes: float64(2), object(1), uint8(1)
memory usage: 706.1+ MB
```

Datetime Information

What the IMCC 2015–2018 dataset represents-

- It’s a classified change layer, not a yearly time series.
- Each pixel encodes whether imperviousness changed between 2015 and 2018, and possibly the type of change (increase, decrease, stable).
- The TIFFs themselves have no time dimension per pixel — the whole

dataset is just a spatial snapshot of change across a fixed period.

So, do the dates matter?

- **Yes, for context/metadata:** Adding `start_date` and `end_date` is useful when you later merge this dataset with others (e.g., soil, climate, socio-economic data). It tells you that these imperviousness values summarize that 3-year window.
- **No, for pixel-level analysis:** Since every pixel shares the same start/end dates, it doesn't add extra information at the row level. It's essentially metadata that applies to the whole DataFrame.

In [ ]:
```
#check the xml file for datetime information

import xml.etree.ElementTree as ET
metadeta_file = '/content/extracted_files/IMCC_1518_020m_hu_03035_v010/Metadat

tree = ET.parse(metadeta_file)
root = tree.getroot()


#Extract all date like fields
dates = []
for elem in root.iter():
  if 'date' in elem.tag.lower():
    dates.append((elem.tag, elem.text))


print("Metadata dates found:")
for tag, val in dates:
  print(tag, ":", val)
```

Metadata dates found:
{http://www.isotc211.org/2005/gmd}dateStamp :

{http://www.isotc211.org/2005/gco}DateTime : 2020-07-10T11:13:37
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2020-07-10
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2020-07-10
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}maintenanceAndUpdateFrequency :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2008-06-01
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2002-03-01
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2010-07-06
{http://www.isotc211.org/2005/gmd}dateType :

```
{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2018-08-16
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2015-07-17
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2019-05-22
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gmd}CI_Date :

{http://www.isotc211.org/2005/gmd}date :

{http://www.isotc211.org/2005/gco}Date : 2010-12-08
{http://www.isotc211.org/2005/gmd}dateType :

{http://www.isotc211.org/2005/gmd}CI_DateTypeCode : None
```

In [ ]:

```python
# Grab all date values
all_dates = []
for elem in root.iter():
    if elem.text and any(tag in elem.tag.lower() for tag in ["date", "datetime
        all_dates.append(elem.text.strip())


print("All dates found in metadata:", all_dates)
```

```
All dates found in metadata: ['', '2020-07-10T11:13:37', '', '', '', '2020-07-1
0', '', '', '', '', '2020-07-10', '', '', '', '', '', '2008-06-01', '', '', '',
'', '2002-03-01', '', '', '', '', '2010-07-06', '', '', '', '', '2018-08-16',
'', '', '', '', '2015-07-17', '', '', '', '', '2019-05-22', '', '', '', '', '20
10-12-08', '']
```

In [ ]:
```python
# ---- Logic to detect observation period ----
# Rule of thumb: observation period falls between ~2015 and ~2018
start_date = None
end_date = None
pub_date = None

for d in all_dates:
  if d.startswith('2015'):
    start_date = d
  elif d.startswith('2018'):
    end_date = d
  elif d.startswith('2020'): #publication date
    pub_date = d

print("\nExtracted Dates:")
print("Start of Observation:", start_date)
print("End of Observation:", end_date)
print("Publication Date:", pub_date)
```

```
Extracted Dates:
Start of Observation: 2015-07-17
End of Observation: 2018-08-16
Publication Date: 2020-07-10
```

In [ ]:
```python
# Add the above in a dictionary

dataset_metadata = {
    "dataset" : "IMCC_1518_020m",
    "start_date": "2015-07-17",
    "end_date": "2018-08-16",
    "publication_date": "2020-07-10"
}
```

IMCC Classes for RGB pixel values

- Check the `symbology` folder and see the classes for each RGB value.
- Map these classes to the dataset as a separate column.

In [ ]:
```python
# Read the symbology folder
# path to file
symbology_file = '/content/extracted_files/IMCC_1518_020m_hu_03035_v010/Symbol

# Read into a dataframe
symbology_df = pd.read_csv(
    symbology_file, header = None,
    names = ["value", "R", "G", "B", "A", "label"]
```

```
)

print(symbology_df)
```

```
   value    R    G    B    A                                                label
0      0  240  240  240  255                          unchanged areas (IMD=0%)
1      1  255    0    0  255                                           new cover
2      2    0  100    0  255                                       loss of cover
3     10  156  156  156  255  unchanged areas (IMD>0% at both reference dates)
4     11  255  191    0  255                                       increased IMD
5     12   64  178    0  255                                       decreased IMD
6    254  255    0  255  255     unclassifiable in any of parent status layers
7    255    0    0    0  255                                        outside area
```

Mapping the classes to the data

In [ ]:
```
# Build mapping dictionaries

# Map numeric values -> labels
label_mapping = dict(zip(symbology_df['value'], symbology_df["label"]))

# Map numeric values -> RGB tuples
color_mapping = dict(zip(symbology_df["value"],
                         symbology_df[["R","G","B"]].apply(tuple, axis=1)))
```

In [ ]:
```
full_df.columns
```

Out[ ]:
```
Index(['x', 'y', 'imperviousness', 'source_file'], dtype='object')
```

In [ ]:
```
# Add descriptive class label
full_df["class_label"] = full_df['imperviousness'].map(label_mapping)

# Add RGB color (as tuple, can be used in plotting)
full_df["rgb"] = full_df["imperviousness"].map(color_mapping)
```

In [ ]:
```
## See a sample of the dataset
full_df.head()
```

Out[ ]:

	x	y	imperviousness	source_file
0	4900410.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
1	4900430.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
2	4900450.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
3	4900470.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif
4	4900490.0	2699990.0	10	IMCC_1518_020m_E49N26_03035_v010.tif

In [ ]:
```
full_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29616769 entries, 0 to 29616768
Data columns (total 6 columns):
 #   Column          Dtype  
---  ------          -----  
 0   x               float64
 1   y               float64
 2   imperviousness  uint8  
 3   source_file     object 
 4   class_label     object 
 5   rgb             object 
dtypes: float64(2), object(3), uint8(1)
memory usage: 1.1+ GB
```

In [ ]:
```
full_df.describe()
```

Out[ ]:

	x	y	imperviousness
count	2.961677e+07	2.961677e+07	2.961677e+07
mean	5.059785e+06	2.753454e+06	7.199314e+01
std	1.697608e+05	1.133539e+05	1.066624e+02
min	4.700010e+06	2.500010e+06	1.000000e+00
25%	4.927230e+06	2.666670e+06	1.000000e+01
50%	5.053330e+06	2.761050e+06	1.000000e+01
75%	5.234390e+06	2.866370e+06	2.550000e+02
max	5.299990e+06	2.899990e+06	2.550000e+02

Save the DataFrame into a separate file

In [ ]:
```
 #Save DataFrame to CSV
imcc_features_csv = '/content/imperviousness_hungary.csv'
full_df.to_csv(imcc_features_csv, index=False)

print(f"✅ DataFrame saved to {imcc_features_csv}")
```

```
✅ DataFrame saved to /content/imperviousness_hungary.csv
```

In [ ]:
```
# Save in parquet format

output_parquet = '/content/imperviousness_hungary.parquet'
full_df.to_parquet(output_parquet, index=False)

print(f"✅ DataFrame saved to {output_parquet} (compressed & faster than CSV)"
```

```
✅ DataFrame saved to /content/imperviousness_hungary.parquet (compressed & fas
ter than CSV)
```

In [ ]:

In [ ]:

In [ ]:

In [ ]: